Software Requirements Specification

for

EventHub

Version 1.0 approved

Prepared by Group-P05-02

Software Engineering: Process and Tools

10/08/2025

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 5
2.7 Assumptions and Dependencies 5
3. External Interface Requirements 5
3.1 User Interfaces 5
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 6
4. Nonfunctional Requirements 7
4.1 Performance Requirements 7
4.2 Safety Requirements 7
4.3 Security Requirements 7
4.4 Software Quality Attributes 7
4.5 Business Rules 8
5. Other Requirements 8
6. System Architecture 9
6.1 Architecture Overview 9
6.2 Architectural Decisions 9
7. User Interface Design 10
Appendix A: Glossary 10
Appendix B: Analysis Models 11
Appendix C: To Be Determined List 11
Appendix D: Milestone 2 Appendix 11

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document covers the software requirements identified for Version 1.0 of EventHub. It provides a description of EventHub's functionalities as well as its associated classes, operating environments and design choices. The scope of the document is EventHub's entire system.

1.2 Document Conventions

This document includes technical terminology which will be listed in the Glossary (Appendix A).

1.3 Intended Audience and Reading Suggestions

The intended readers of this document are the EventHub developers and the project's client, who make up the entirety of the team associated with the product.

The rest of this document contains details of the product and its structure. Those desiring an overall explanation of the product should read section 2 for an overview of context, functions, assumptions and constraints. For developers, more technical descriptions regarding user, hardware, software and communication interfaces can be found in section 3, along with key quality attributes in section 4. Developers should also refer to the system architecture diagram and overview in section 6, which sets the structure of the overall system. The client should refer to section 7 for a rough idea of what the finished product might look like in terms of its user interface.

1.4 Product Scope

EventHub is a web application which enables university clubs and organisations to create and manage events. These events can then be browsed by students who can indicate attendance and interest. The overall goal of EventHub is to increase the visibility of student-run events, promoting community engagement and participation within universities.

1.5 References

[1] K. E. Wiegers, "SRS Template - IEEE," 1999. Available: https://www.studocu.com/in/document/silicon-institute-of-technology/software-engineering/sample-srs-document/46613267.

2. Overall Description

2.1 Product Perspective

EventHub, is a new self-contained, centralized event management web application addressing the need for a unified event platform within the university environment. Through consolidation of fragmented channels (e.g., social media, email, bulletin boards) to promote and manage events, EventHub aims to become the central hub for all campus-related events.

The system will function independently, but for enhanced functionality may integrate with:
- Third party APIs such as Google Maps, Google Calendar
- Public REST APIs for listing events and submitting RSVPs

The system is intended as the primary digital solution for event coordination within the university and is not a direct upgrade from any in-house tool. As such, it can be seen as a replacement for the many existing disjointed and informal communication methods.

2.2 Product Functions

User Registration & Authentication
- Create new user accounts (student, organizer, admin)
- Manage profile

Home / Welcome Page
- Display list of upcoming and featured events
- Recommended events list
- Quick access to search, filters and RSVP history

Event Management
- Create, edit and delete events.
- Add or modify event details: title, description, date/time, location, category, keywords
- Manage event visibility and searchability: tags, images

User Participation
- Browse and search events
- RSVP to events and view RSVP history
- Submit feedback after attending events
- Share event details via social media or copied links

Event Recommendations
- Suggest and display events based on user profile and history

Event Gallery
- Upload photos after an event
- Share or link galleries

Help / Support Page
- Frequently asked questions
- Help topics
- Contact information

Administrative Tools
- Moderate events and delete inappropriate content
- Manage user accounts
- Access platform-wide reports and analytics

Extension Features
- Calendar view for events
- QR code check-in
- CSV export of attendees
- Achievements for user engagement
- Public REST API integration with external apps

Online Payment
- Secure payment gateway integration
- Support for multiple payment methods (credit/debit cards, PayPal, Apple Pay)
- Automatic email receipt and invoice generation

2.3 User Classes and Characteristics

This system will serve three main user groups:

1. **Students**
   Description:
   - Primary audience / general users, university students' interest in attending events.
   - Seek to browse, RSVP, receive recommendations and relevant event updates.

   Key Characteristics:
   - Frequency: Moderate to high (several times per week during semesters).
   - Technical Expertise: Basic to Intermediate
   - Privilege Level: Standard
   - Educational Level: Tertiary Students

   Functions Accessed:
   - View event listings and details
   - RSVP to events and view history
   - Submit event feedback
   - Manager personal profile

   Importance:
   - High

2. **Organizers**
   Description:
   - Leaders of student clubs, societies, or organizations responsible for creating and managing events
   - Relevant staff members of the university
   - Seek a centralized, single platform that can both manage and promote events

   Key Characteristics:
   - Frequency: High (during event planning and immediately after).
   - Technical Expertise: Intermediate

- Privilege Level: Elevated
- Educational Level: Tertiary Students, University Staff

Functions Accessed:
- Create, edit and delete events
- Upload images to event galleries
- Track attendance and export CSV lists
- Implement attendance rules (18+, club membership)
- Manage payments

Importance:
- High

3. **Admin**
Description:
- University appointed staff or approved members responsible for overseeing platform moderation, compliance and maintenance.
- Ensure event quality, manage disputes, handle user issues, collect platform feedback, ensure compliance and safety.

Key Characteristics:
- Frequency: Daily to weekly.
- Technical Expertise: Advanced
- Privilege Level: Full System Access
- Educational Level: University Staff or IT personnel

Functions Accessed:
- View and manage all events
- Deletion privileges
- Manage user accounts (ban, deactivate, assign roles)
- Access reports and analytics

Importance:
- High

2.4 Operating Environment

The software will be operating on laptops and personal computers, operating systems are MacOS Monterey, Sequoia and Windows 10/11. The application is not resource or graphics intensive so hardware is not a major concern. The application is self-contained and will use a database which will be in SQL.

2.5 Design and Implementation Constraints

The system is to be fully functional before the 13th of October, therefore there is a constraint of time. There are other constraints such as specific tools, where the use of SQL and java is required and also a money constraint, as there is no available budget for the system.

2.6 User Documentation

The primary goal of the system is to help users manage, find and create events. As a result the system will be designed to be as simple and intuitive as possible. Therefore this software will not come with user documentation.

2.7 Assumptions and Dependencies

Assumptions:

- We assume that there are no malicious third parties that wish to steal all our user data
- Our user base will be small, so scalability of the database is not at the for-front of development goals
- Developers will ensure that they are all developing on the latest version of the project and otherwise work on a separate branch/local commit if a new feature requires other already implemented code that may be changed while this feature is developed.
-

Dependencies:

- Third party QR code generator for event attendees
- Third Party payment processing api to test purchases
- Access to google maps api to embed the location of events
- Github will host both the codebase as well as the project board for the planning and development of the project.
- Docker allows the program to be easily deployed as well as isolated from other programs running

3. External Interface Requirements

3.1 User Interfaces

GUI Standards

- Consistency in color schemes, icons and typography across all screens

Screen Layout Constraints

- Navigation bar will appear on the top of the page and will be available across all screens
- Footer will appear on the bottom of the page and will be available across all screens
- The layout of all pages from top of the page to bottom will be as follows: Navigation Bar -> Content (that varies with each page) -> Footer

Standard Buttons and Functions

- Help button on every screen on the navigation bar
- Save button available where relevant (such as saving details of an event as an event organizer)
- Cancel button available where relevant (such as canceling changes to details of an event as an event organizer)
- Edit button available where relevant (such as editing an event as an event organizer of that event)
- Search bar available where relevant (such as searching events)

Keyboard Shortcuts

- Tab: Depending on if you clicked on a specific section of a page (navigation bar, parts of content or footer), it will scroll through the contents of the specific sections

- Enter: commit entry (search, save, cancel, etc.)

Error Message Display Standards

- Red text displaying the code of error and details of the error

3.2 Hardware Interfaces

Input

- User input is captured through keyboard inputs through the web browser
- User input is captured through mouse inputs through the web browser

Output

- Graphical user interface rendered by a web browser
- Web browser is displayed on a physical screen

3.3 Software Interfaces

Operating Systems

- Google Chrome (139.0.7258.67)
- Safari (18.6)
- Mozilla Firefox (141.0.3)
- Opera (120.0.5543.161)
- Microsoft Edge (Version 139)

Databases

- MySQL (v8.4)

Libraries & Tools

- Code Repository: GitHub
- Task Management: GitHub Projects
- Communication: Microsoft Teams
- Documentation: Word, Excel (on Microsoft Teams)
- CI/CD: GitHub Actions
- Java: Version 17 or later
- Web Framework: Spring Boot
- Build Tool: Maven (latest stable version)
- Unit Testing: JUnit 5
- Containers: Docker

3.4 Communications Interfaces

Communication Standards

- HTTP: Communication protocol for client and server interaction
- REST API: Data exchange between frontend and backend elements

Communication Functions

- Web browser: Send / receive requests using HTTP
- Email: Login, signup and password reset

Message Formatting

- API: JSON format
- Sending Emails: Plaintext

Communication, Security and Encryption Issues

- No encryption, Authentication is handled backend logic
- Transfer rates should be able to handle high loads and spikes of traffic

4. Nonfunctional Requirements

4.1 Performance Requirements

System Speed
- Pages should load within 2 seconds
- Submitting information to the database should take 1 second

Resources
- The product should not be graphically intensive
- The product should not be memory intensive
- The product should not be CPU intensive

4.2 Safety Requirements

Data Backup
- All data is backed up daily
- All backup data is stored separately from the main system

Error Handling
- Upon a system error occurring, a clear error message should be displayed to the user
- Minor errors should not stop usage of the site

Data Validation
- Incorrect or conflicting data should be rejected

Real Life Safety
- Usage of the product should hold no physical dangers to the user beyond the danger of using any average website.
- Users should be made aware of the potential dangers of any event that they may sign up for.

4.3 Security Requirements

User Authentication
- Only registered users can log into the platform
- Login requires a username and password

Authorization
- Only authorized users can create and edit events
- Unauthorized users can only view events and RSVP
- Unauthorized users cannot see features they cannot use

Data Protection
- All sensitive information is encrypted when stored

Session Management
- Users are automatically logged out after 15 minutes of inactivity
- Multiple failed logins locks the account temporarily

4.4 Software Quality Attributes

- Usability: For users, the product must be easy and intuitive to navigate, with little need for explanation for how to complete basic tasks on the website (such as viewing events, creating events and joining events).

- Maintainability: For developers, ensuring the codebase maintains a sense of proper order and structure means fixing and or developing new features ensures it is not a troublesome task that could unintentionally break other aspects of the site. (Updating the Event Suggestions portion of the codebase should not in any way affect the Feedback Forum section)
- Testability: For developers, creating dummy accounts from both the user perspective and organizer perspective is simple allowing for quick testing of various site functionalities. (Creating events, joining events etc)
- Security: For developers, this means ensuring that user data is protected/obfuscated and not easily accessible to those who should not be able to view them. (Password/Usernames are not kept on site in plaintext)
- Performance efficiency: For users, the site should load quickly, with page buttons feeling and acting responsive with little to no slowdown between actions (creating an event should feel snapping, feeding back into usability). For developers this means that potentially computationally heavy tasks such as database operations should be optimized where possible. (Featuring a user's login details should not take over 10s)

4.5 Business Rules

Users can be split into two categories: attendees and organizers. Once a user has signed up for the site they become an attendee by default, but if they wish to create events, they also become an organizer. An organizer can do everything an attendee can, alongside some extra roles.

- **Attendees can:**
  - View events
  - RSVP events
  - Attend events (only for RSVP'd events)
  - Leave feedback for an event, a comment and a rating (only those they've attended)

- **Organizers can:**
  - Create events
  - **For events they have made:**
    - Edit or delete events
    - View a list of attendees that wish to attend an event
    - Delete any inappropriate comments
    - Export a list of these attendees (only if there is at least one attendee)

- **Admins can:**
  - Ban/Delete users
  - View user information including what events they are attending
  - **For ALL events:**
    - Edit event details
    - Delete events (inappropriate or illegal events)

5. Other Requirements

Internationalization Requirements

- English is the only supported language on the platform

Legal Requirements

- Contents of the platform are in accordance with Australian law

6. System Architecture

EventHub's system architecture will be a Layered Monolithic Architecture. Due to the nature of the project, the team believes that the numerous benefits outweigh the drawbacks of Layered Monolithic Architecture. For instance, the architecture seems suitable due to the size of the product, being a small to medium sized application. As client specifications are defined and unlikely to change, drawbacks such as difficulty to scale and change the application are greatly mitigated. Additionally, considering the team structure, separating the application into layers allows for leveraging of each member's independent strengths, enabling a more efficient development cycle.

Properties of Layered Monolithic Architecture
Benefits:
- Simple architecture suitable for small to medium applications
- The complexity of the product can be separated and managed into layers (business logic layer, presentation layer, data access layer, reports and analytics layer etc.
- Efficient communication between layers
- Layers allow for specialized teams (or team members)

Drawbacks:
- Tight coupling between layers means a change in one layer may affect all others
- Difficult to scale independent layers due to tight coupling, scaling one layer might require scaling all
- Single point of failure means that a failure in any part of the application could bring down the whole system

6.1 Architecture Overview

6.2 Architectural Decisions

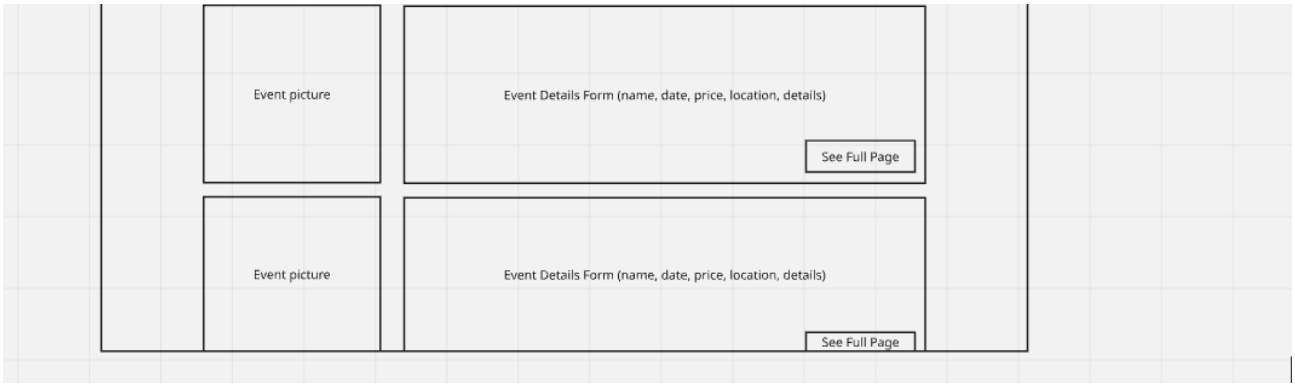
Decision: System Architecture	
Status	Accepted
Date	18/08/25
Context	Deciding on an overall system architecture before beginning development
Decision	Layered Monolithic Architecture
Consequences	- Product now separated into layers, allowing for easier allocation of tasks and leveraging of team member strengths. - Efficiency of communication between components increased as all are within the same process - Relatively simple architecture, enables all stakeholders and team members to understand and conform - Significant changes (technology, frameworks) to any layers may be more difficult as it may affect the entire application

	- Difficulty to scale any single layer as it may require scaling the entire application - Single point of failure as failure in any layer might cause failure in the entire application.

Decision: Web App Framework	
Status	Accepted
Date	18/08/25
Context	Deciding on a web app framework before beginning development
Decision	Springboot
Consequences	- Reduction in boiler plate code, allowing for the team to work on product specific features - Pre-built components allow for building prototypes quickly - Increased client satisfaction as client has a preference for Springboot per the specifications

Decision: Backend Database	
Status	Accepted
Date	19/08/25
Context	Deciding on a database before beginning development
Decision	SQL Database
Consequences	- SQL databases are designed to handle large volumes of data and are generally highly performant - SQL is industry standard so can enable ease of use and accessibility - Team experience on SQL leads to faster development and team coherence

7. User Interface Design



Appendix A: Glossary

OS: Operating System
Database: an electronically stored, systematic collection of data.
Event: In the context of EventHub, an event is a student club planned public or social occasion.

HTTP: Hypertext Transfer Protocol; Enables connection between web browsers and web servers.
API: Application Programming Interface
REST API: An API that follows the rules of Representational State Transfer architecture style.

Appendix B: Analysis Models



Appendix C: To Be Determined List

References – TBD

Appendix D: Milestone 2 Appendix

New Requirements

- Connection to MySQL database